

SISTEMAS DIGITALES

Linux Sockets Tutorial

Working with Linux Sockets

Sockets lie in the foundation of networking; within a network node of a computer network they provide an endpoint for sending and receiving data across the network. There are two compressed directories in the accompanying archive: `02-sockets` and `02-sockets-PingPong`.

1. Simple server and client

Once the archive is uncompressed change to the `02-sockets` directory. Inside it there is a `Makefile`. Use `make` to compile the source code in that directory. The first pair of programs you should execute in a concurrent manner are called `server1` and `client1`. Hence you will need to use two terminals, one per program.

The server program creates a socket (`server_sockfd`) and prepares it for logical connections from any client (`INADDR_ANY`) using the TCP protocol (`AF_INET`, `SOCK_STREAM`) on port `SERVER_PORT` (using `bind`). Then, the server will establish a queue whose depth can hold up to `MAX_CLIENTS` (as specified in `listen`); subsequent attempts to connect will be refused.

The server then enters an endless loop in which it just blocks (on `accept`) waiting for a connecting client; once a client connection is accepted (via `accept`), a different socket (`client_sockfd`) is used for exchanging data with the client. Thus, the server blocks (on `read`) waiting for a character. When a character arrives from the client via `client_sockfd`, the server increments the ASCII value of the received character, turns it back (using `write`) to the client via `client_sockfd`, and then closes the `client_sockfd` (and the connection). Notice that once a socket is closed, it can be reused in new connections. Notice also that every connection uses

On the client side, the program creates a socket (`sockfd`) and prepares it to connect to a server on IP v4 address `127.0.0.1`, destination port `SERVER_PORT`. This particular IP address corresponds to the `localhost` address, and identifies the machine on which the client as well as the server (in this case) are running. If the client runs on a machine that is different from that of the server, on the server machine we can use the `ifconfig -a` command to get its `eth0` IP address. Then, on the client side we can use that address in the `SERVER_ADDR` definition.

Then the client asks the user for a character, sends it to a local server (using `write`) via `sockfd` and blocks (on `read`) waiting for the server response; then the client echoes to the screen the answer from the server and closes the `sockfd` socket (and the connection).

You may run more clients (one per terminal) to see how the use of blocking functions and a queue

allow the concurrent execution of several clients.

You can kill the server by pressing `<Ctrl>-<C>` or use the `killall` command:

```
killall server1
```

You can also try the `client2` program against `server1`. `client2` can connect to a computer whose IP address is given as an argument in the command line. To determine the IP address of the machine that is running the server, you can use the `getIPs` program in the target machine where the server will run (use the `compile` script to generate the binary image). Notice that `client2` and `server1` can run in different computers.

Enhanced server and client

The next pair of programs to test are called `server2` and `client2`. Now the server creates a new process image via `fork`, so that the interaction with each client is served by a spawned process while the server itself keeps waiting for new clients. The client, when invoked with no parameters attempts to connect to a server running in the same machine in which the client executes; the host name is obtained via `gethostname`. With this client it is possible to use a *FQDN* for the server. In this case, you need to pass its name in the command line; the client uses the `gethostbyname` function to obtain some network information for the server, including its IP address. It is necessary to use the `inet_ntoa` function to convert the IP address to the format required by the `inet_addr` function.

Examining the state of TCP connections on the system

We can see any active TCP network connections by using the `netstat` (or the `ss`) command:

```
netstat -t or ss -t -a
```

2. Simplified echo protocol

Change to the `02-sockets-PingPong` directory. Inside it there is a `Makefile`. Use `make` to compile the source code in that directory. The directory has two programs, called `ping.c` and `pong.c`. `ping` and `pong` exchange messages via sockets. As before, you will need to use two terminals, one per process.

The server process (`pong`) waits for a connection on port `SERVER_PORT`. Just 1 client is allowed to connect simultaneously. The server enters a loop in which it waits for an incoming message; the contents of the message is an integer that denotes the sequence number (`SeqNo`) of a data packet. Each arriving message is turned back to the client. The loop executes five times. After that, the server closes the sockets.

The client (`ping`) connects to the server and sends five messages. Each message has an associated sequence number (`ICMP_req`), which can be used to differentiate each message. For that purpose, each time a message is sent, its sequence number (`ICMP_req`) is incremented by 1; the first message has sequence number 1. Since `pong` executes the server role, it should be executed first.